

Alta Disponibilidad, Balanceo de Carga y Replicación de BD en Linux

Ing. Olaf Reitmaier Veracierta

Caracas, Venezuela
Enero de 2008 - Julio 2009

Shared Disk Failover

- Completamente activado por hardware.
- Una sola copia de la BD => 0% de sincronización.
- Disco (Arreglo) compartido por varios servidores.
- Falla Serv. Principal (Activo) => Servidor en Espera (Pasivo) pueden montar e iniciar la base de datos (Recovery After Crash).
- Recuperación rápida sin pérdida de datos.
- Mecanismo intrínseco de una SAN o NFS (POSIX).

Shared Disk Failover

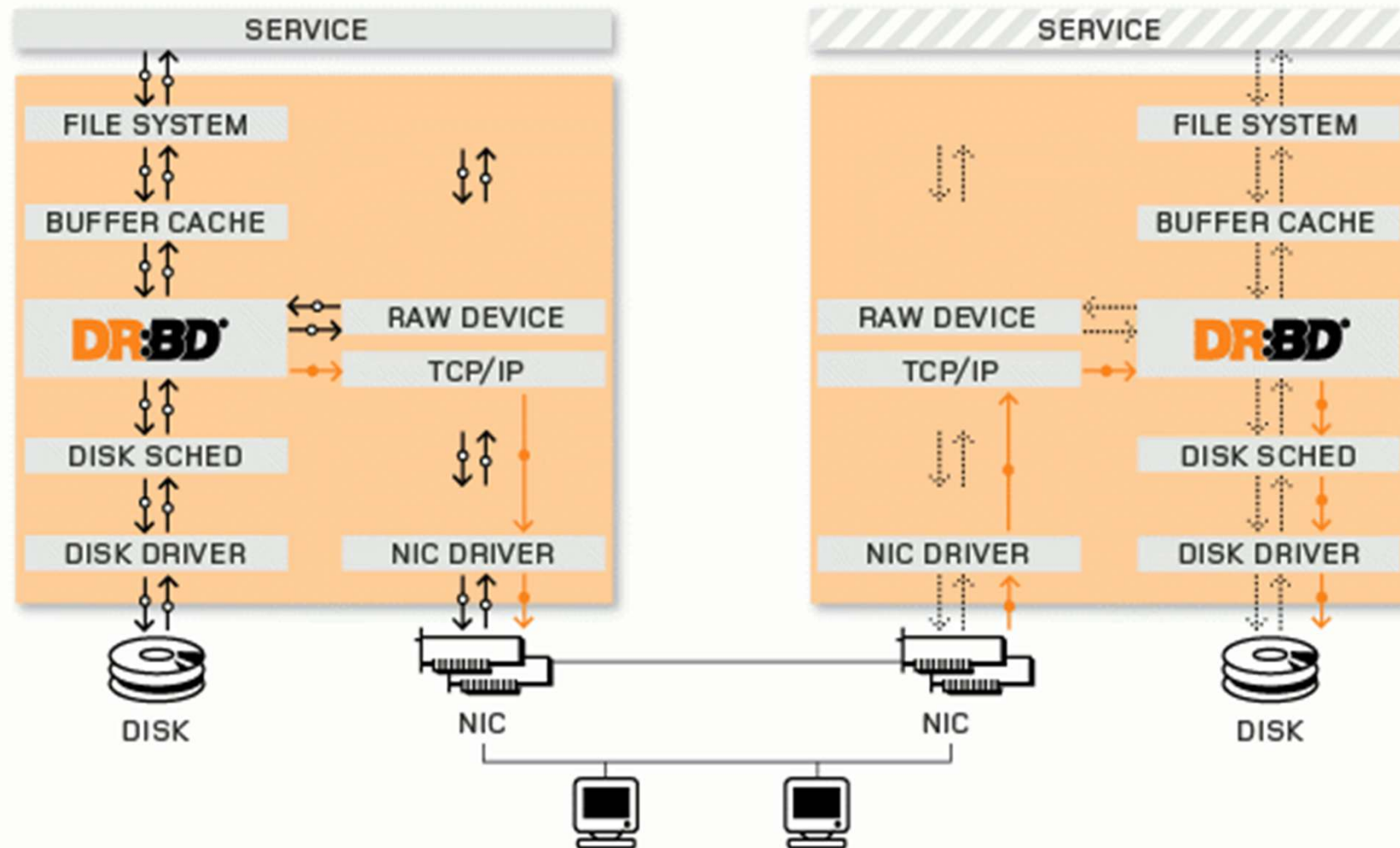
- **Desventajas:**

- Si el disco falla o se corrompe todos los servidores primarios quedan inhabilitados.
- Si el servidor primario está activo, el servidor en espera NO puede acceder al mismo tiempo al disco compartido.

File System Replication

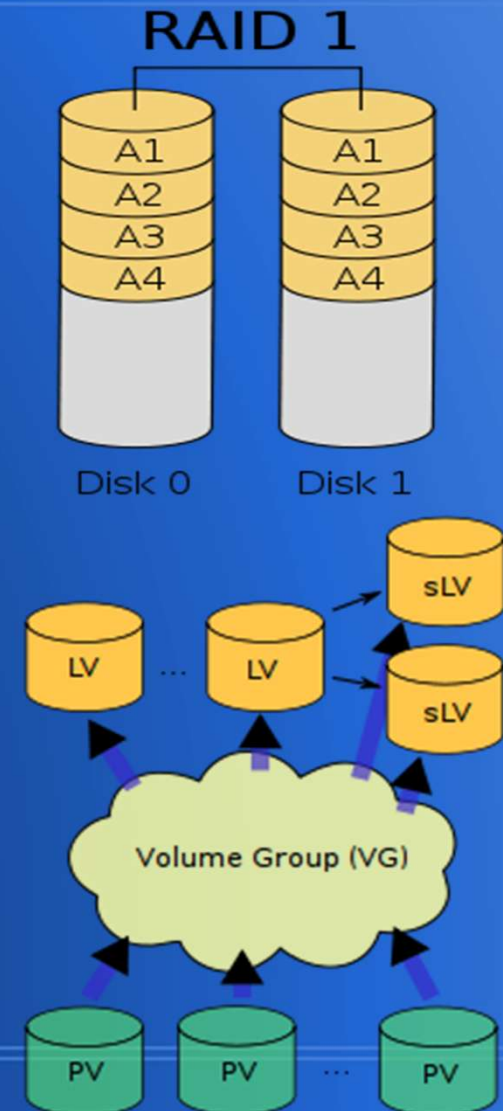
- File System == Block Device?.
- Es un versión modificada de SDF que consiste en replicar los cambios de un FS en servidor a otro FS en otro servidor.
- La replicación de los sistemas debe ser siempre consistente (en Orden).
- Distributed Replicated Block Device (DRBD) es una solución para Linux.

DRBD - Estructura



DRBD - Requisitos

- Red Gigabit Ethernet dedicada.
- Ancho de banda dedicado (Conf. 30%).
- Ejecutar `apt-get install drbd`.
- Dos (2) nodos OK / tres (3) nodos no recomendado.
- Replicación en RAID 1.
- **Resize y/o Shrinking en línea.**
- Documentación y soporte muy buenos.
- Single (v7.0) o Dual (v8.0) (Multimastering).
- **Compatible con LVM2, GFS y OCFS2.**



DRBD – Conf. + Adm. I

/etc/drbd.conf

```
global { usage-count yes; }
common { protocol C;}
resource replica0 {
  on nodo1 {
    device /dev/drbd0;
    disk /dev/sda7;
    address 192.168.1.5:7789;
    meta-disk internal;
    on-io-error detach;
    rate 37.5M;
  }
  ...
}
```

Por nodo (n veces):

```
drbdadm create-md replica0
drbdadm attach replica0
drbdadm syncer replica0
drbdadm connect replica0
cat /proc/drbd
drbdadm disconnect replica0
drbdadm dettach replica0
```

En el primario (1 vez):

```
drbdadm -- --overwrite-data-of-peer \
primary replica0
```

Intercambio de roles:

```
drbdadm primary replica0
drbdadm secondary replica0
```

Estado:

```
drbdadm cstate replica0
drbdadm dstate replica0
drbdadm role replica0
```

DRBD – Conf. + Adm. II

/etc/drbd.conf

```
global { usage-count yes; }
  common { protocol C;}
  resource replica0 {
    on nodo1 {
      device /dev/drbd0;
      disk /dev/sda7;
      address 192.168.1.5:7789;
      meta-disk internal;
      on-io-error detach;
      rate 37.5M;
    }
    ...
  }
```

Verificación de datos (crontab):

```
drbdadm verify replica0
```

Resincronización:

```
drbdadm disconnect replica0
drbdadm connect replica0
```

Cambio de tasa de replicación:

```
drbdsetup /dev/drbdnum syncer -r 110M
drbdadm adjust resource
```

DRBD - Estrategias Errores I/O

- **detach:** Recomendado, detiene la replicación y pasa a modo diskless.
- **pass_on:** Reporta error de I/O en capas superiores. En el servidor primario se reporta a nivel de sistema de archivos montados. En el servidor secundario, es ignorado porque éste no tiene capa a donde reportarlo. Este es el predeterminado por razones históricas pero no es el recomendado.
- **call-local-io-error:** Invoca el comando definido como el manejador de errores I/O locales, el cual, se define a discreción del administrador.

DRBD - Consideraciones - LVM

- Logical Volume Manager (LVM): Versiones 1 y 2. Abstracción de la estructura de dispositivos de bloques en Linux.
- LVM Implementa sólo RAID 0 no RAID 1 ni 5, se recomienda usar software/hardware específico para esto.
- Redimensionado en caliente de grupos de... y volúmenes lógicos.
- LVM anidado, CLVM ó GFS, se deben cambiar los filtros de reconocimiento de dispositivos (devices) en `/etc/lvm/lvm.conf`.
 - `filter = ["a|sd.*|", "a|drbd.*|", "r|..*|"]`
- Si se utiliza CLVM ó GFS, en `/etc/lvm/lvm.conf` se debe añadir:
 - `locking_type = 3`

DRBD - Consideraciones - GFS

/etc/drbd.conf

```
resource resource {  
    startup {  
        become-primary-on both;  
        ...  
    }  
    net {  
        allow-two-primaries;  
        after-sb-0pri discard-zero-changes;  
        after-sb-1pri discard-secondary;  
        after-sb-2pri disconnect;  
        ...  
    }  
    ...  
}
```

- **Global File System (GFS)** es simplemente un sistema de archivos de lectura/escritura concurrente generalmente usado con LVM:

En la consola:

```
mkfs -t [gfs|gfs2] -p lock_dlm -j 2  
    /dev/vg-name/lv-name
```

En el /etc/fstab:

```
/dev/vg-name/lv-name mountpoint  
[gfs|gfs2] defaults 0 0
```

DRBD - Consideraciones - OCFS2

/etc/drbd.conf

```
resource resource {  
    startup {  
        become-primary-on both;  
        ...  
    }  
    net {  
        #allow-two-primaries;  
        after-sb-0pri discard-zero-changes;  
        after-sb-1pri discard-secondary;  
        after-sb-2pri disconnect;  
        ...  
    }  
    ...  
}
```

- Oracle File System v2 (OCFS2) es simplemente un sistema de archivos de lectura/escritura concurrente específico para base de datos Oracle.

...

Configurar Oracle...

...

mount -t ocfs2 /dev/drbd0 /shared

PITR en PostgreSQL

- Warm Standby Using Point-In-Time Recovery (PITR), Warm Standby or Log shipping, permiten alta disponibilidad.
- Servidor en espera en caliente lee un pedazo de una bitácora de escritura adelantada (Write Ahead Log) de registros.
- Si el servidor principal falla, el servidor en espera en caliente contiene casi todos los datos del servidor principal, y puede convertirse rápidamente en el servidor principal.

PITR en PostgreSQL

- La bitácora debe ser atómica (Obviamente).
- Las operaciones son asíncronas y sólo pueden realizarse para toda la base de datos (no hay sincronizaciones parciales).
- Generalmente se implementa para base de datos.
- PostgreSQL soporta esta forma de funcionamiento

PITR en PostgreSQL

- Log Shipping = Mover o copiar los (WAL) desde un servidor a otro (Principal hacia Espera en Caliente).
- Log Shipping es asíncrono debido a que debe esperar por los commit de las transacciones de la base de datos (ACID):
 - **Atomicidad, Consistencia, Aislamiento, Durabilidad.**
- Archivo WAL pesa 16 MB, la tasa de transferencia depende de las transacciones en el servidor principal.

PITR en PostgreSQL

- Log Shipping por registro (Si se necesita una ventana de tiempo i.e. minuto) y sustituir el esquema por bloques de 16 MB.
- En un fallo catastrófico las transacciones no enviadas (shipped) se perderán, configurar `archive_timeout` = mínimo posible/requerido en segundos (mientras más bajo mayor ancho de banda será necesario).
- El servidor en espera en caliente no está disponible para consulta porque siempre está realizando un proceso de recuperación basado en los archivos WAL

PITR en PostgreSQL

- El proceso de recuperación es super-eficiente y el servidor en caso de ponerse en producción sólo tardará unos segundos (actualización continua).
- Restaurar un servidor desde un respaldo basado en archivos de respaldo a disco puede tardar horas.
- PITR es una estrategia de alta disponibilidad no recuperación de desastre.

PITR en PostgreSQL

- Hardware puede ser distinto, pero la experiencia dice lo contrario, es más fácil de administrar cuando son iguales.
- Puntos de montaje de TABLESPACES deben existir y ser los mismos en los servidores principales y en espera en caliente.
- Versiones mayores distintas de PostgreSQL no está permitido (ie. 7.4, 8.3).

PITR en PostgreSQL

- Cuidado de no mezclar archivos WAL de diferentes servidores primarios en los servidores en espera en caliente.
- PITR no permite detectar si el servidor primario falla, existe otras técnicas como IP round-robin, etc.
- PITR es una solución de alto nivel TRX SQL basado en una especie de REDO LOG (Oracle) llamado WAL.

PITR en PostgreSQL

Ejemplo (pg_standby):

```
Llamado = false;
while (!HayWAL() && !Llamado)
{
    sleep(100000L); /* ~0.1 seg */
    if (ChequearLlamado())
        Llamado = true;
}
if (!Llamado)
    CopiarYRecuperarWAL();
```

PITR + pgSQL - Configuración

- Configurar el servidor primario y el(los) servidor (es) en espera en caliente.
- Configurar el almacenamiento del archivo WAL en el servidor principal hacia una carpeta de (los) servidores en caliente (pg_standby) en el archivo **“postgresql.conf”**
 - `archive_mode = true`
 - `archive_command = 'cp -i %p /mnt/server/archivedir/%f </dev/null'`
 - `archive_timeout = 30 # segundos`
- Hacer un respaldo del servidor principal y restaurarlo en el (los) servidor(es) en espera en caliente.
- Iniciar la recuperación de los archivos WAL en el (los) servidor(es) en espera en caliente configurando el archivo `reconver.conf` que especifique un `restore_command`.

PITR + pgSQL - Consideraciones

- Todo archivo WAL es de sólo lectura para los servidores en espera en caliente. Por lo tanto, puede ser respaldado en cinta.
- Es posible ejecutar ambos el servidor principal y el (los) servidor(es) en espera en caliente en el mismo servidor.

PITR + pgSQL - Recuperación

- Si el servidor principal falla el servidor en espera en caliente debe iniciar las labores de recuperación por falla (failover procedure).
- Si el servidor secundario falla entonces no se requiere realizar ningún procedimiento de recuperación por falla.
- Si el servidor falla momentáneamente, y luego, reinicia sus labores, debe configurarse un mecanismo para notificarse, es decir, hacer un STONITH (Shoot the Other Node in the Head).

PITR + pgSQL – Respaldo Inc.

NombreC	Tipo	Descripción
pg_start_backup(label text)	text	Ejecuta un respaldo en línea
pg_stop_backup()	text	Termina un respaldo.
pg_switch_xlog()	text	Obliga a cambiar a nuevo archivo de log.
pg_current_xlog_location()	text	Obtiene el nombre del archivo de log actual.
pg_current_xlog_insert_location()	text	Obtiene el lugar de inserción actual del archivo de log actual.
pg_xlogfile_name_offset(location text)	text, integer	Convierte a una cadena de caracteres la ubicación y el offset del archivo de log actual.
pg_xlogfile_name(location text)	text	Convierte a una cadena de caracteres la ubicación del archivo de log actual.

Respaldo Inc. – PITR + pgSQL

- Determinar último WAL a repaldar:
 - Conectarse a la base de datos en Prueba01 como superusuario (postgres o algún superuser).
 - **SELECT pg_start_backup('respaldo01');**
 - Esperar todo lo que sea necesario hasta que se cree un Checkpoint (Si igual que en Oracle y el REDO log).
 - **SELECT * FROM pg_xlogfile_name_offset(pg_stop_backup());**
 - **file_name: 0100000D**
 - **file_offset: 4039624**
 - El nombre del archivo es **\$file_name\$file_offset**
- Copiar el WAL en su último estado:
 - **cp -i pg_xlog/0100000D4039624
/mnt/server/archivedir/0100000D4039624 < /dev/null**

Master Slave Replication

- Se ha implemetado como esquema de replicación de alto nivel (bases de datos, SQL, etc.) no de bajo nivel (discos, arreglos, sistema de archivos, etc.)
- Las modificaciones primarias son escritas en el servidor Maestro.
- El servidor maestro se encarga de transferir las modificaciones a los servidor esclavos de forma asíncrona.

Master Slave Replication

- El servidor esclavo acepta consultas de sólo lectura todo el tiempo concurrentemente.
- El servidor esclavo es perfecto para operaciones de Data-Warehouse y Data-Mining.
- En PostgreSQL se implementa con Slony-I, en MySQL es nativo y sencillo de implementar.

Slony + pgSQL

- Enterprise Level Replication System.
- Slon = Elefante en Ruso, Slony = Elefantes.
- Granularidad por Tabla.
- Actualización batch de los esclavos (múltiples)
- Esclavos en cascada (Alimentación descendente).
- Se vale de Multi-Version Concurrency Control (MVCC) de PostgreSQL.
- Posible pérdida de datos si hay fallo catastrófico.

Slony + pgSQL - Cálculos

- Comunicaciones en factor cuadrático $n(n-1)$, TCP, SYNC como respuesta BACK OK, un total de $n(n/2)$, es decir, $n/2$ SYNC por esclavo.
- Recomendado PostgreSQL 8.3, todas las bases de datos DEBE TENER EL MISMO ENCODING.
- Todos los servidores deben tener implementado Real Time Clock vía NTP.
- Subred es la configuración más fácil y recomendada.
- Ejecutar `apt-get install slony1*`

Slony + pgSQL - Conceptos

- **Cluster:** Conjunto de instancias PostgreSQL.
- **Node:** Nodo Slony-I.
- **Replication Set:** Conjunto de tablas y secuencias a replicar e un Cluster Slony-I
- **Origin, Providers, Suscribers:** Cada replicación tiene un origen, donde se realizan las modificaciones de las tablas replicadas (Master Provider). Los demás nodos se suscriben al cluster para recibir datos. El nodo origen nunca será considerado un suscriptor (Pero pueden haber cascadas de clusters).
- **Slon daemon:** proceso que maneja el cluster en cada nodo, hecho en C, maneja los eventos de configuración y de sincronización.
- **Slonik configuration processor:** es una línea de comandos con un lenguaje especial que permite configurar el cluster.

Slony + pgSQL - Conceptos

- Origin:
 - Esquema de BD: cbcluster
- Suscribers:
 - Esquema de BD: _cbcluster
 - Número de Nodo: Único e Inmutable.
- Pasos para definir un Replication Set:
 - Definir claves candidatas para las tables a replicar que no tienen clave primaria.
 - Definir las tablas que serán replicadas.
 - Definir las secuencias que serán replicadas.

Slony + pgSQL - Conceptos

- Origin:
 - Esquema de BD: cbcluster
- Suscribers:
 - Esquema de BD: _cbcluster
 - Número de Nodo: Único e Inmutable.ç
- Configuración: Terrible.
- Administración: Terrible.
- Desempeño: SQL para replicar SQL?.

Statement Based Replication Middleware

- También llamado Statement Based Replication Middleware (SBRM).
- Intercepta todas las instrucciones SQL (Lectura/Escritura) y las envía a todos los servidores.
- Cada servidor trabaja independientemente, pero juntos hacen un manojo (pool).
- Se puede segmentar las instrucciones de sólo lectura a sólo algunos de los servidor del manojo, permitiendo el balanceo de la carga de lectura.
- Si las consultas se envían si modificación a los servidores `CURRENT_TIMESTAMP` serán diferentes en todos los servidores.

Statement Based Replication Middleware

- Todas las consultas deben hacer commit o rollback en todos los servidores, esto implica que se debe utilizar un commit de dos (2) fases.
- Permite paralelización de consultas (partes ejecutadas en varios servidores), lo cual, implica particionar las tablas.
- SBRM se implementa con pg-pool II y sequoia.

Pg-pool II – Matriz de Conf.

Función/Modo	Modo Básico	Modo Pool de Conexiones	Modo Replicación	Modo Maestro Esclavo	Modo Consultas Paralelas
Pool de Conexiones	Si	Opcional	Opcional	Opcional	Opcional
Replicación	Si	Si	Opcional	Si	(*)
Balanceo de Carga	Si	Si	Opcional	Opcional	(*)
Degeneración	Si	Si	Opcional	Opcional	Si
Recuperación por falla	Opcional	Opcional	Si	Si	Si
Paralelización de Consultas	Si	Si	Si	Si	Opcional
Número mínimo de servidores	1 o más	2 o más	2 o más	2 o más	2 o más
Base de datos de Sistema Requerida.	No	No	No	no	
Autenticación	Trust, Plain Text, md5, crypt, PAM	Trust, Plain Text, md5, crypt, PAM	Trust, Plain Text, PAM	Trust, Plain Text, PAM	Trust, Plain Text, md5, crypt, PAM

(*) = No se puede utilizar para las tablas particionadas en el modo paralelo de consultas.

Pg-pool II

- Ejecutar `apt-get install pgpool2`
- Editar el archivo `/etc/pgpool.conf`, colocar
 - `pcp_port = 9898`
- Editar el archivo `/etc/pool_hba.conf`, parecido al `pg_hba.conf` de postgresql.
- `/etc/init.d/pgpool2 start | stop | restart`
- Utilizar `pg_md5` para generar una contraseña para postgres en `/etc/pcp.conf`
- Online Recovery similar a PITR, con pequeñas sutilezas en la configuración.

Pg-pool II

- Definir los nodos en pgpool.conf

```
backend_hostname0 = 'localhost'
```

```
backend_port0 = 5432
```

```
backend_weight0 = 1
```

```
backend_hostname1 = 'localhost'
```

```
backend_port1 = 5433
```

```
backend_weight1 = 1
```

```
backend_hostname2 = 'localhost'
```

```
backend_port2 = 5434
```

```
backend_weight2 = 1
```

Pg-pool II

- Definir los modos de replicación:
 - Distribuir consultas entre todos los nodos:
 - `replication_mode = true`
 - Distribuir consultas `SELECT` entre todos los nodos:
 - `load_balance = true`

Pg-pool II – Chequear la Replicación

- A través de pgpool:
 - createdb -p 9999 bench_replication
 - pgbench -i -p 9999 bench_replication

- Contra cada nodo:

```
for port in 5432 5433 5434; do
    echo $port
    for table_name in branches tellers accounts history; do
        echo $table_name
        psql -c "SELECT count(*) FROM $table_name" \
            -p $port bench_replication
    done
done
done
```

Pg-pool II – Modo Paralelo

- En postgresl.conf:

listen_addresses = '*'

- En pgpool.conf:

parallel_mode = true

replication_mode = false

load_balance_mode = true

system_db_hostname = 'localhost', system_db_port = 5432

**system_db_dbname = 'pgpool', system_db_schema =
'pgpool_catalog', system_db_user = 'pgpool', system_db_password
= ''**

- Como postgres o superusuario:

createuser -p 5432 pgpool

createdb -p 5432 -O pgpool pgpool

Pg-pool II – Modo Paralelo

- Habilitar los dblinks:
 - `USE_PGXS=1 make -C contrib/dblink`
 - `USE_PGXS=1 make -C contrib/dblink install`
 - `psql -f /usr/local/pgsql/share/contrib/dblink.sql -p 5432 pgpool`
- Definir la tabla de distribución `dist_table`:
 - `psql -f /usr/share/pgpool-II/system_db.sql -p 5432 -U pgpool pgpool`
- Definir las reglas de distribución (Ver manual).
- Definir las reglas de replicación (Ver manual).

Pg-pool II – Chequear Paralelización

- A través de pgpool (Scaling factor 3):
 - createdb -p 9999 bench_parallel
 - pgbench -i -s 3 -p 9999 bench_parallel

- Contra cada nodo:

```
for port in 5432 5433 5434; do
    echo $port
    for table_name in branches tellers accounts history; do
        echo $table_name
        psql -c "SELECT min(aid), max(aid) \
            FROM accounts" -p $port bench_parallel
    done
done
```

Pg-pool II – Restricciones en la Replicación

Funciones:

- No hay garantía de que los metadatos tales como: número aleatorios, TRX Id, OID, SERIALES, SECUENCIAS, CURRENT_TIMESTAMP sean replicados correctamente.
- Estas restricciones deben y puede ser manejadas generalmente por el programador a nivel de aplicación.

Pg-pool II – Restricciones en la Paralelización

- No... INSERT INTO t(x) VALUES (DEFAULT);
- No... INSERT INTO t(x) VALUES (func());
- No... SELECT INTO
- No... INSERT INTO ... SELECT
- INSERT debe tener constantes en la clave de particionamiento.
- No se deben actualizar las Claves (Primarias) de las tablas particionadas (No hay particionado automático).

Pg-pool II – Restricciones en la Paralelización

- No... COPY *
- No... SELECT ... FOR UPDATE
- ALTER/CREATE TABLE, si se cambian las reglas de particionado debe reiniciarse pgpool para que las cargue.
- SELECT dentro de transacciones serán ejecutados en una transacción aparte.
- No... JOIN entre nodos.
- VIEW sólo entre tablas del mismo nodo, particionadas o no, ciertas condiciones aplican.

Pg-pool II – Restricciones en la Paralelización

- El protocolo de consultas extendidas de JDBC no es soportado por pgpool.
- El encoding del cliente, la base de datos y la base de datos del sistema debe ser la misma... Ups... LATIN1 != UTF8
- No... Consultas con múltiples instrucciones.
- No se detectan deadlock para SELECT ... FOR UPDATE. Mejor no utilizar.

Pg-pool II – Restricciones en la Paralelización

- Los objetos fuera del esquema publico deben ser calificado completo (i.e. esquema.objeto), el esquema correcto no puede ser determinado si se ha utilizado `set search_path = xxx` y no se especifica el esquema para cada objeto de la consulta.
- El nombre de una tabla o columna no puede empezar por `pool_`, cuando pgpool procesa el SQL sustituye el valor por una constante especial.

Pg-pool II – Restricciones de la Base de datos del Sistema

- Sólo se puede definir una columna clave de particionado para una tabla, condiciones tales como 'x' or 'y' no se permiten.
- El cache de consultas debe ser eliminado manualmente, no se invalida el cache cuando nuevos datos son almacenados.

Pg-pool II – Comandos

- Argumentos de los comandos pg-pool:

Time Out en segundos, Servidor, Puerto, Usuario
PGP, Contraseña PGP.

- Lista de comandos:

- pcp_node_count, pcp_node_info, pcp_proc_count,
pcp_proc_info, pcp_systemdb_info,
pcp_detach_node, pcp_attach_node,
pcp_stop_pgpool

Asynchronous Multimaster Replication

- Para servidores que no están constantemente en línea (Servidores remotos, laptop, etc.) mantener la data consistente es todo un reto.
- Periódicamente los servidores se comunican entre sí y resuelven los conflictos a través de las reglas programadas por los usuarios (o redes neuronales).
- No hay una implementación práctica.

Synchronous Multimaster Replication

- Algunas implementaciones utilizan discos compartidos para evitar al sobrecarga de la red.
- No hay necesidad de particionar las cargas de trabajo entre los servidores maestros y esclavos.
- Tampoco existen problemas con las funciones no determinísticas porque los cambios son replicados entre todos.
- PostgreSQL no ofrece esto, sin embargo, puede utilizar el esquema de transacciones PREPARADAS para implementarlo.

Synchronous Multimaster Replication

- Todos los servidores aceptar peticiones de escritura pero los cambios son transmitidos a todos los servidores antes de hacer COMMIT.
- Las consultas de lectura pueden ser enviadas a cualquier servidor.
- Mucha escritura puede causar bloqueos, lo cual, trae como consecuencia bajo desempeño en todos los nodos. De hecho, en este ambiente las escrituras funciona peor que un servidor sólo.

Soluciones Comerciales

- Debido a que PostgreSQL es de código abierto bajo la licencia BSD, muchas empresas han tomado su código y generado muchas soluciones comerciales con características especiales.

Comparación de Alternativas

Es trategia	Shared Disk Failover	File Sys tem Replication	Warm Standby Us ing PITR	Master-Slave Replication	Statement-Based Replication Middleware	As ynchro no us Multimas ter Replication	Synchr onous Multimas ter Replication
No se requiere HW especial		•	•	•	•	•	•
Multimaster					•	•	•
No hay Sobrecarga del Master	•		•		•		
No hay espera por múltiples servidores	•		•	•		•	
Fallo del servidor master nunca produce pérdida de datos.	•	•			•		•
Esclavos aceptan consultas de lectura.				•	•	•	•
Granularidad Por Tabla				•		•	•
No se necesita resolver conflictos	•	•	•	•			•
Método de comunicación.	shared disk	disk blocks	WAL	table rows	SQL	table rows	table rows and row locks

Alta Disponibilidad, Balanceo de Carga y Replicación de Sitios Web en Linux

Ing. Olaf Reitmaier Veracierta

Caracas, Venezuela
Enero de 2008 - Julio 2009

HA + LB Web Cluster

Apache 1:

192.168.0.101

Apache 2:

192.168.0.102

Balanceador 1:

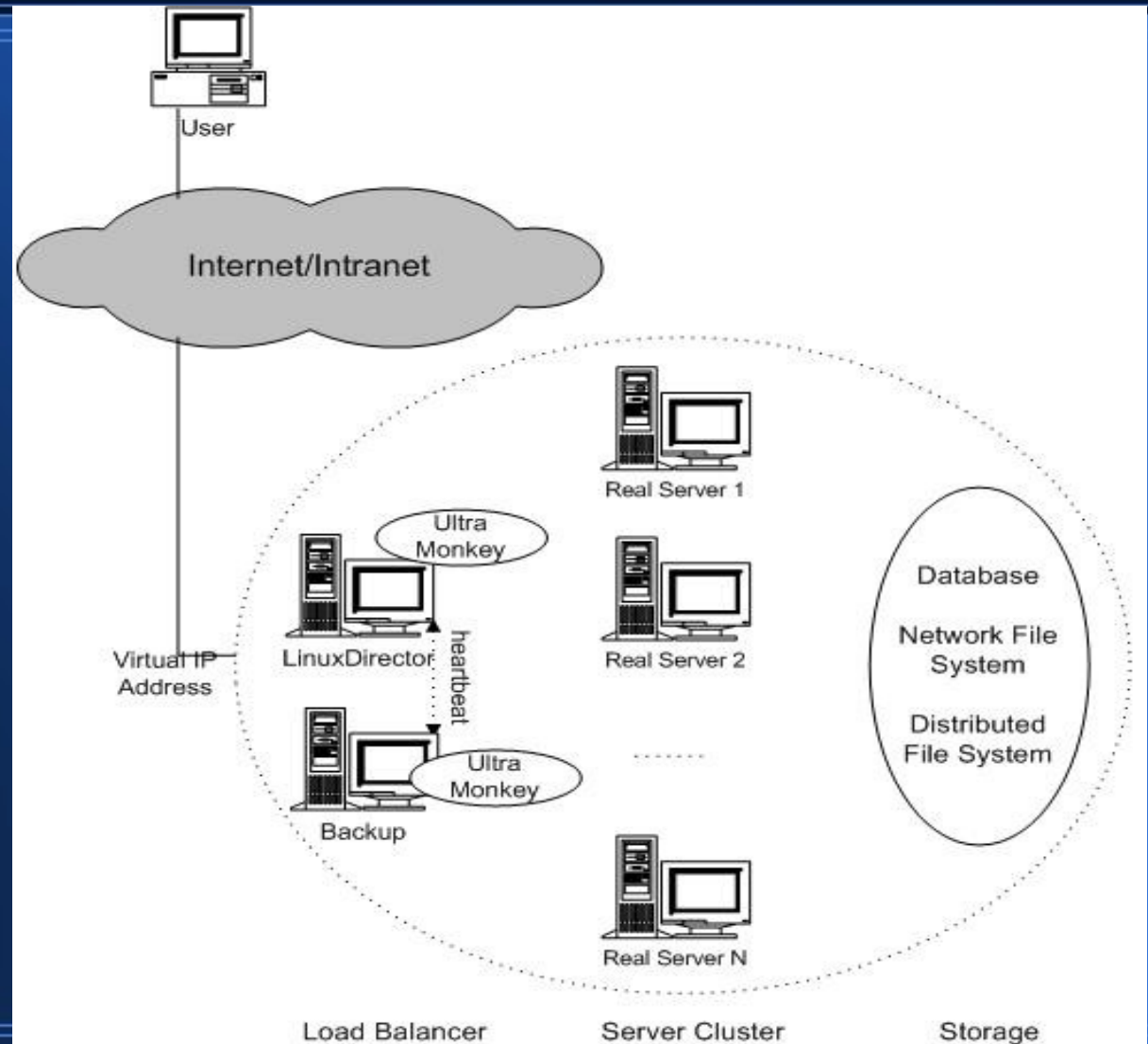
192.168.0.103

Balanceador 2:

192.168.0.104

IP Virtua:

192.168.0.105



HA + LB Web Cluster

- Balanceadores (lb1, lb2):
 - Ejecutar `apt-get install ipvsadm`
Rules on boot? No
Daemon method? None
 - Habilitar el reenvío de paquetes
Editar `/etc/sysctl.conf`
`net.ipv4.ip_forward = 1`
Ejecutar `sysctl -p`

HA + LB Web Cluster

- Balanceadores:
 - Ejecutar apt-get install heartbeat
 - Nombres lb1 y lb2 son iguales a **uname -n**
 - Editar el archiv /etc/ha.d/ha.cf:

```
logfacility    local0
bcast        eth0
mcast eth0 225.0.0.1 694 1 0
auto_failback off
node         lb1
node         lb2
respawn hacluster /usr/lib/heartbeat/ipfail
apiauth ipfail gid=haclient uid=hacluster
```

HA + LB Web Cluster

- Balanceadores:

- Editar el archivo /etc/ha.d/authkeys

auth 3

3 md5 "somerandomstring"

- Editar el archiv /etc/ha.d/haresources:

lb1 \

ldirectord::ldirectord.cf \

LVSSyncDaemonSwap::master \

lpaddr2::192.168.0.105/24/eth0/192.168.0.255

HA + LB Web Cluster

- Balanceadores:
 - Ejecutar apt-get install ldirectord

```
Checktimeout=10
checkinterval=2
autoreload=no
logfile="/local0"
quiescent=yes
```

```
Virtual=192.168.0.105:80
real=192.168.0.101:80 gate
real=192.168.0.102:80 gate
fallback=127.0.0.1:80 gate
service=http
request="/ldirector.html"
receive="Test Page"
scheduler=rr
protocol=tcp
checktype=negotiate
```

HA + LB Web Cluster

- Balanceadores (Salidas On/Off):
 - Ejecutar
`ldirectord ldirectord.cf status`
 - Ejecutar
`ipvsadm -L -n`
 - Ejecutar
`/etc/ha.d/resource.d/LVSSyncDaemonSwap master status`

HA + LB Web Cluster

- Servidores Web Apache
 - Ejecutar apt-get install iproute
 - Editar /etc/sysctl.conf

net.ipv4.conf.all.arp_ignore = 1

net.ipv4.conf.eth0.arp_ignore = 1

net.ipv4.conf.all.arp_announce = 2

net.ipv4.conf.eth0.arp_announce = 2

HA + LB Web Cluster

- Editar /etc/sysctl.conf

auto lo:0

iface lo:0 inet static

address 192.168.0.105

netmask 255.255.255.255

pre-up sysctl -p > /dev/null

- Ejecutar if up lo:0
- Editar /var/www/ldirectord.html

HA + LB Web Cluster

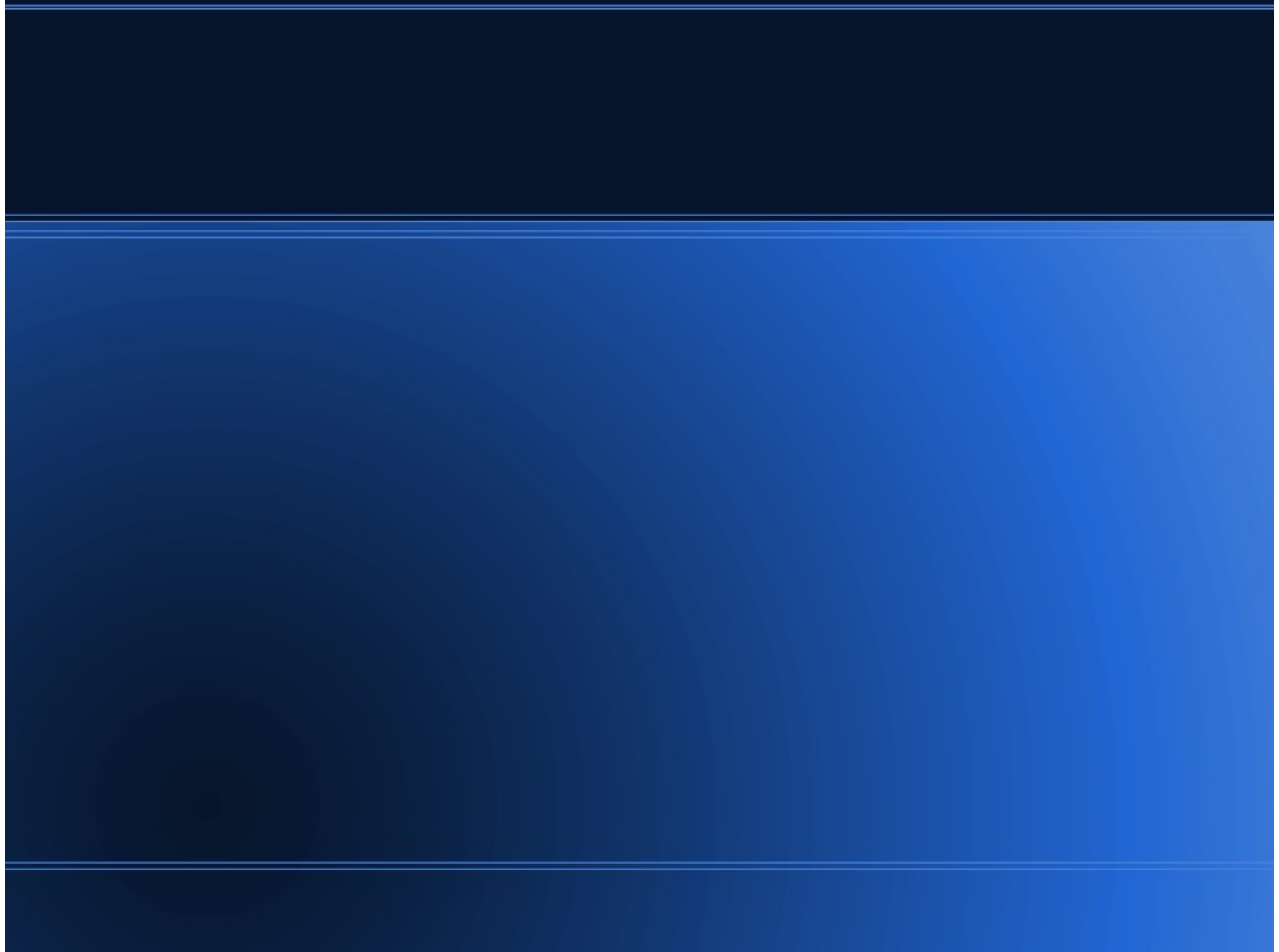
- Balanceadores (Activo/Pasivo):
 - Ejecutar `/etc/init.d/heartbeat ñstop | start`

HA + LB Web Cluster

- Configurar el almacenamiento / consulta de las sesiones de PHP (Tomcat) en la BD.
 - Crear tabla de sesiones + Crear UUID.
 - Definir handler y/o clases de almacenamiento.
- PHP:
 - <http://www.developertutorials.com/tutorials/php/saving-php-session-data-database-050711/page1.html>
- Tomcat:
 - <http://tomcat.apache.org/tomcat-5.5-doc/config/manager.html>
 - <http://www.fwd.at/tomcat/sharing-session-data-howto.html>

Solución Final

- Cluster de Base Datos
 - SAN + PG-POOL II + pgSQL Cluster
- Cluster de Aplicaciones Web
 - IPVS + iproute + heartbeat + Ldirectord



Gracias

Referencias

<http://www.postgresql.org/docs/8.3/static/high-availability.html>

<http://es.wikipedia.org/wiki/LVM>

<http://www.postgresql.org/docs/current/interactive/warm-standby.html>

<http://es.wikipedia.org/wiki/ACID>

<http://pgpool.projects.postgresql.org/pgpool-II/doc/pgpool-en.html>

<http://archives.postgresql.org/pgsql-es-ayuda/2005-11/msg00856.php>

<http://www.postgresql.org/docs/current/interactive/sql-commit-prepared.html>

<http://www.postgresql.org/docs/current/interactive/sql-prepare-transaction.html>

<http://pgpool.projects.postgresql.org/pgpool-II/doc/tutorial-en.html>

<http://www.drbd.org/users-guide/ch-heartbeat.html>

<http://www.drbd.org/docs/install/>

<http://www.slony.info>

<http://projects.postgresql.org>

<http://pgfoundry.org/>